

04 - Modules and Packages

哈尔滨工业大学 黄子扬

2026 年 6 月 6 日

Contents

1	Standard Library / 标准库核心模块	2
1.1	Copy Module / 复制模块	2
1.2	Math & Time Modules / 数学与时间	2
2	External Libraries & Pip / 外部库与包管理	2
2.1	Pip Manager / Pip 管理工具	2
3	Numerical Computing with NumPy / NumPy 数值计算	3
3.1	Why NumPy? / 为什么 NumPy 这么快?	3
3.2	Anatomy of Arrays / 数组解剖	3
3.3	Array Operations / 数组运算逻辑	3
4	Aggregation & Axis / 聚合函数与轴逻辑	4
5	Combining Arrays / 数组合并	4

1 Standard Library / 标准库核心模块

1.1 Copy Module / 复制模块

【Concept / 核心概念】1

Assignment vs. Copy / 赋值与复制的区别 Assignment (=) does not copy objects; it creates a binding between a name and an object.

赋值 (=) 不复制对象，只是起别名。修改一个会影响另一个 [1]。

- **Shallow Copy / 浅拷贝**：创建一个新容器，但其中的元素仍引用旧对象 [1]。
- **Deep Copy / 深拷贝**：彻底递归复制所有内容，完全摆脱与原对象的关联 [1]。

【Example / 实操案例】1

Copying Example / 复制案例

```
import copy
animals = ["lion", "tiger"]
# Independent copy without references
large_cats = copy.copy(animals)
large_cats.append("Leopard")
# Now animals remains ["lion", "tiger"] [2, 3]
```

1.2 Math & Time Modules / 数学与时间

- **math**：提供常数 pi, e 及函数 log, ceil (向上取整), floor (向下取整) [1, 3]。
- **time**：time.time() 返回自 1970 年以来的秒数；time.sleep(2) 让程序暂停 2 秒 [3, 4]。

2 External Libraries & Pip / 外部库与包管理

2.1 Pip Manager / Pip 管理工具

【Concept / 核心概念】1

Installing Packages External packages are not included by default. Use pip, a command line tool, to install them [4, 5].

【Example / 实操案例】1

Common Commands / 常用命令

```
pip install numpy           # Install the latest version
pip list                    # List all installed packages
pip uninstall numpy         # Remove a package [6, 7]
```

3 Numerical Computing with NumPy / NumPy 数值计算

3.1 Why NumPy? / 为什么 NumPy 这么快？

【Concept / 核心概念】1

Type-checking Bottleneck / 类型检查瓶颈 Python lists can store any object, so the interpreter must check the data type for every operation. NumPy uses **ndarrays** with a fixed data type, skipping these checks and using vectorized operations [7, 8].

Python 列表因为支持异构数据，每次计算都要检查类型。NumPy 通过固定数据类型（**dtype**）并使用向量化计算，速度快几十倍 [8, 9]。

3.2 Anatomy of Arrays / 数组解剖

- **dtype**: Returns the data type of the array elements [10].
- **shape**: A tuple with the number of elements in each dimension (e.g., (3, 4)) [11].
- **ndim**: The total number of dimensions [11].

【Example / 实操案例】1

Creating Arrays / 创建数组

```
import numpy as np
a = np.arange(2, 14, 2)           # Like range: [2, 4, ..., 12]
b = np.linspace(-5, 5, 9)        # 9 values from -5 to 5 [12, 13]
c = np.zeros((2, 2))             # 2x2 matrix of zeros [12, 13]
```

3.3 Array Operations / 数组运算逻辑

【Concept / 核心概念】1

Vectorization / 向量化 Operations are applied to each element without explicit for-loops [11].
不需要写循环，直接对整个数组进行数学运算。

【Warning / 避坑指南】1

Matrix Multiplication / 矩阵乘法 Use **arr * arr** for element-wise multiplication.
Use **arr @ arr** for the **matrix product** (inner product for 1D) [14, 15].

4 Aggregation & Axis / 聚合函数与轴逻辑

【Concept / 核心概念】1

The Axis Argument / 轴参数的精髓 Aggregation functions (min, max, sum) reduce dimensionality [14].

- **axis=0**: Operation applied across rows (column-wise reduction) / **纵向** (按列计算) [16, 17]。
- **axis=1**: Operation applied across columns (row-wise reduction) / **横向** (按行计算) [14]。

【Example / 实操案例】1

Masking / 掩码过滤 Logical arrays (masks) can be used to filter data [2].

```
arr = np.arange(1, 6)
mask = arr > 3          # [False, False, False, True, True]
print(arr[mask])        # Output: [18, 19] [17, 20]
```

5 Combining Arrays / 数组合并

- **np.concatenate**: Joins a sequence of arrays [16, 17].
- **np.stack**: Stacks arrays along a **new** axis [16, 17].